

SIMATS ENGINEERING

Assessment -CO1-AT3

CO1: Apply AI basics such as intelligent agents and

uninformed search methods to solve problems effectively. (BL3)

Subject	Artificial Intelligence
Course Code	CSA 1712
Name	Vignesh Kumar K
Register Number	192512375

Declaration

I, Vignesh Kumar K (Reg No: 192512375), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Question 1 — Intelligent Agents in a Smart Home Automation System

Scenario: A smart home system manages lighting, temperature, and security based on user behaviour and environmental conditions, responding to real-time inputs (motion, temperature, time of day) while learning user preferences over time.

1. Simple Reflex Agent

Acts only on the current percept using fixed condition–action rules, with no memory of the past.

In this system: e.g. 'IF motion detected THEN turn ON light', 'IF temperature > 26°C THEN turn ON AC'. It reacts instantly to sensor triggers.

Limitation: has no memory, so it cannot distinguish a room that is still occupied (person sitting still, no motion) from an empty one — lights may switch off on someone mid-task. It also cannot learn preferences or anticipate needs, since every decision is based purely on the immediate percept.

2. Model-Based Reflex Agent

Maintains an internal state/model of the world that is updated as new percepts arrive, allowing it to reason about aspects of the environment that are not directly observable at every instant.

In this system: keeps track of 'room recently occupied' status (with a timeout window rather than switching off the instant motion stops), current time-of-day context, and door/window lock status — enabling smoother, context-aware behaviour than a pure reflex agent.

Limitation: still driven by fixed condition–action rules over its internal state; it does not explicitly reason about goals or weigh trade-offs (e.g., comfort vs. energy cost).

3. Goal-Based Agent

Chooses actions by reasoning about which action sequences will achieve explicit goals, rather than reacting via fixed rules.

In this system: goals such as 'maintain room temperature within the comfort range', 'ensure the house is secure whenever unoccupied', or 'pre-cool the house before the user's predicted arrival time' can be actively planned for — the agent can decide the sequence of heating/cooling or locking actions needed to reach a target state, not just react to the current percept.

Limitation: goals are typically treated as satisfied/unsatisfied; it does not naturally handle situations where multiple goals conflict (e.g., exact comfort temperature vs. minimizing energy use) or need to be balanced by degree.

4. Utility-Based Agent

Extends the goal-based agent with a utility function that scores how desirable each possible state/outcome is, allowing it to choose among several goal-satisfying options by picking the one with the highest overall utility.

In this system: combines comfort, energy efficiency, and security into a weighted utility score, e.g. $Utility = w_1 \cdot ComfortScore - w_2 \cdot EnergyCost + w_3 \cdot SecurityScore$, and picks the action that maximizes it — for instance, choosing a slightly less precise but far more energy-efficient temperature setting when the difference in comfort is minor. It can also adjust its weights over time based on observed user overrides, which is exactly how the system 'learns user preferences over time'.

Comparative Summary

Agent Type	Internal Structure	Decision Basis	Strength	Limitation
Simple Reflex	None	Reacts to current percept only	Fast, simple	No memory; poor with partial observability; can't learn or trade off goals
Model-Based Reflex	Internal state model	Rules applied to state + percept	Handles partial observability, more context-aware	Still rule-based; no explicit goal reasoning or trade-off handling
Goal-Based	State + explicit goals	Plans actions to achieve goals	Flexible; goals can change without rewriting rules	Cannot weigh/trade off multiple simultaneous goals
Utility-Based	State + goals + utility function	Maximizes a weighted utility score	Balances competing objectives; supports learning/adapting preferences	More complex to design and tune (weights/utility function)

Recommendation and Justification

Recommended design: a **Model-Based Utility-Based Agent** (i.e., a utility-based agent built on a model-based internal-state architecture, with its utility weights adapted through observed user behaviour over time).

- Model-based foundation is necessary because the environment is only partially observable at any instant (motion sensors detect movement, not continuous presence), so the system needs memory/state to avoid erratic reflex-driven behaviour.
- Goal-based reasoning is needed to pursue explicit objectives such as maintaining comfort ranges and ensuring security when the house is unoccupied.
- Utility-based evaluation is essential because comfort, energy efficiency, and security frequently conflict (e.g., maximum comfort costs more energy) — only a utility function can systematically balance these rather than treating each goal as a rigid yes/no target.
- The utility weights can be continuously refined from user feedback (manual overrides, usage patterns), which is precisely how the system 'learns user preferences over time' as required by the scenario.
- Security, however, is best treated as a near-hard constraint layered underneath the utility optimizer (e.g., never unlock or disable alarms purely to save energy) — combining goal-based guarantees for

safety-critical behaviour with utility-based optimization for comfort/energy trade-offs gives the most robust overall design.

Question 2 — Robot Navigating an Unknown Maze: Uninformed Search and Agent Design

Scenario: A robot is placed in an unknown maze and must find a path to the exit with no prior knowledge of the layout.

1. How Uniform Cost Search (UCS) Helps

UCS models the maze as a weighted graph — each cell is a node and each move between cells has an associated cost (uniform, e.g., 1 per step, or variable if some maze cells represent harder terrain). UCS expands the frontier node with the lowest cumulative cost $g(n)$ using a priority queue, guaranteeing the **least-cost path** to the exit is found, even when different moves have different costs.

Because the maze layout is initially unknown, classical UCS (which assumes the whole graph is known in advance) cannot be applied directly — it must be adapted into an **online form**: the robot senses its immediate surroundings, adds newly discovered cells/edges to its internally built graph, and applies UCS-style lowest-cumulative-cost expansion only over the currently known portion of the maze, repeating this sense–update–expand cycle until the exit is reached.

2. How Iterative Deepening Search (IDS) Helps

IDS performs a series of depth-limited depth-first searches, increasing the depth limit by one each iteration, until the exit is found. It combines DFS's low memory footprint with BFS's completeness and (for uniform step costs) optimality — and crucially, it does **not require knowing the maze's depth or size in advance**, making it naturally suited to an unknown maze: the robot simply explores a little further with each iteration, backtracking when it hits a dead end within the current depth limit.

3. Advantages and Disadvantages (Time and Space Complexity)

Algorithm	Time Complexity	Space Complexity	Advantages	Disadvantages
Uniform Cost Search	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ — exponential in the ratio of optimal cost C^* to the smallest step cost ϵ	Same order as time complexity — stores the full frontier and explored set	Guarantees the least-cost path even with varying step costs; complete when all costs are positive	High memory usage for large/deep mazes; in an unknown environment it must be run online (sense-then-expand), adding overhead; slow when cost differences are large
Iterative Deepening Search	$O(b^d)$ — same asymptotic order as BFS/DFS; repeated shallower iterations	$O(b \cdot d)$ — linear in depth, since it behaves like DFS at each iteration	Very low memory usage; complete and optimal for uniform step costs; ideal when	Regenerates nodes across iterations (some redundant computation); does not account for

Algorithm	Time Complexity	Space Complexity	Advantages	Disadvantages
	add only a constant-factor overhead		maze size/depth is unknown in advance	varying step costs — only guarantees the fewest-steps path, not the cheapest, unless all moves cost the same

4. Relating the Problem to Agent Types and How Agent Design Affects Decision-Making

Since the robot has no prior map and can only sense its immediate surroundings, this is fundamentally an **Online Search Agent** problem, not a classical offline search — the robot must interleave sensing, planning, and acting rather than compute a complete solution before moving (offline uninformed/informed search assumes a fully known graph, which does not exist here).

Underlying this online behaviour, the robot also needs a **model-based** internal architecture: it must maintain an internal map of visited/known cells (marking dead ends, visited nodes, and discovered passages) to avoid endlessly re-exploring the same areas.

Effect of agent design on decision-making:

- A memoryless (simple-reflex-style) agent applying uninformed search would effectively wander blindly, re-visiting the same cells repeatedly since it retains no record of where it has already been — this is far slower and can even fail to terminate efficiently in cyclic maze layouts.
- A model-based agent that builds and updates an internal map as it explores can apply systematic strategies (backtracking on dead ends, IDS-style depth-limited exploration, or online UCS-style cost-aware expansion) far more efficiently, since it never repeats a fully explored dead-end branch.
- This shows that the choice of agent architecture is not a side detail but directly determines how effectively a given search strategy can actually be executed in a real, only-partially-known environment.

5. Recommended Combination and Justification

Recommended: a Model-Based Online Search Agent, choosing the search strategy based on whether the maze has uniform or variable movement costs:

- If every move has the same cost (a typical grid maze): use online Iterative Deepening Search. It is complete and optimal (fewest steps) for uniform-cost mazes while using only linear $O(b \cdot d)$ memory — ideal for a robot with limited onboard memory exploring an unknown space of unknown size.
- If different cells/terrain carry different traversal costs (e.g., rough or hazardous sections): use an online form of Uniform Cost Search (a real-time, cost-aware search such as LRTA*-style expansion). Only a cost-based strategy like UCS can guarantee the genuinely cheapest exit path — IDS alone cannot, since it only minimizes the number of steps, not the total cost.

In both cases, pairing the chosen strategy with a **model-based agent architecture** (so the robot remembers visited cells, dead ends, and the map it has built so far) is essential — without it, neither algorithm can be executed efficiently in an environment that is discovered incrementally rather than known in advance. This

combination — **model-based online agent + IDS (uniform cost) or online UCS (variable cost)** — best balances completeness, optimality, and the strict memory/computation limits of a real physical robot exploring an unknown maze.